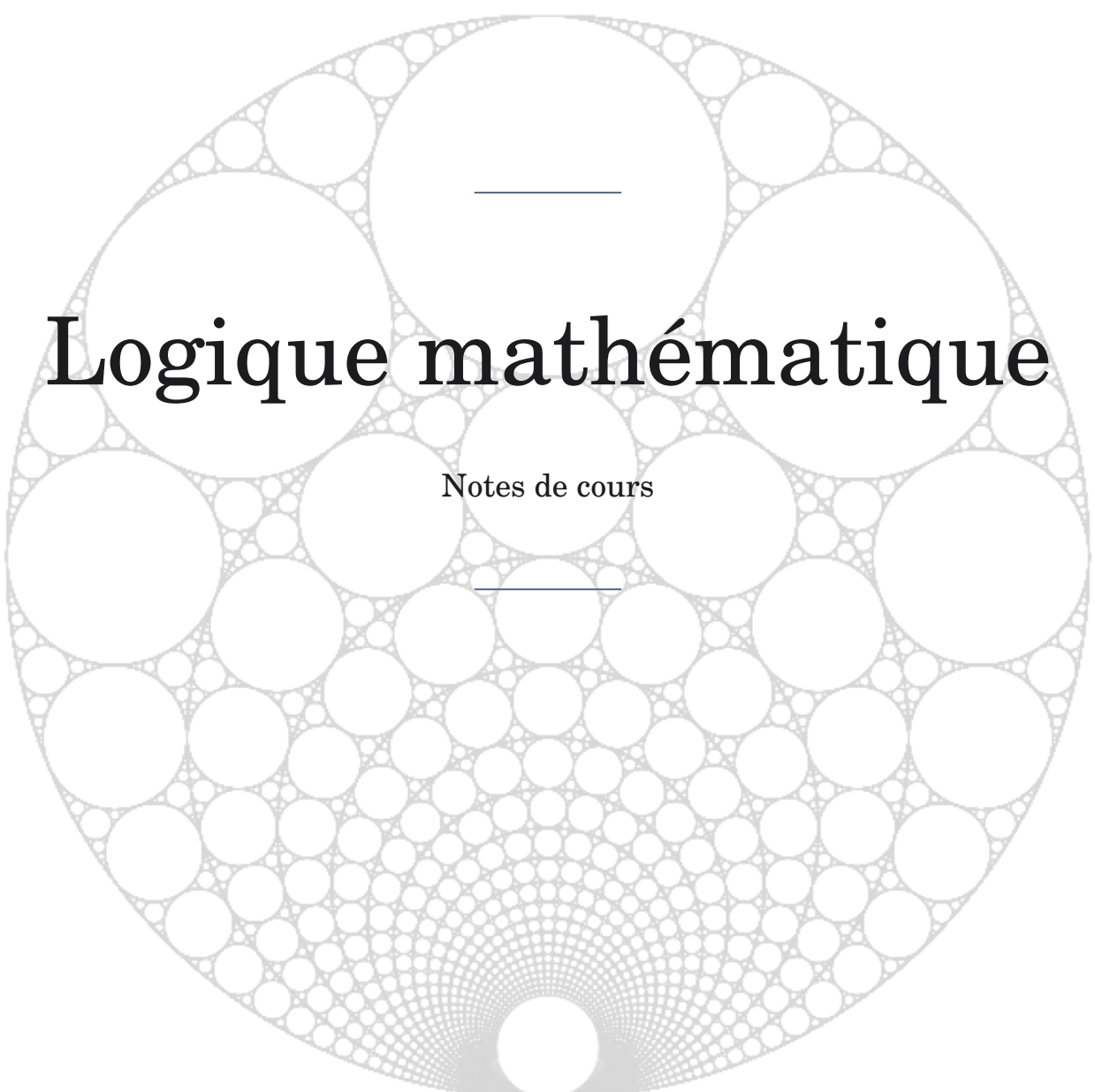


UNIVERSITÉ DE MONS · FACULTÉ DES SCIENCES



Logique mathématique

Notes de cours

TIAGO PIETTE

BaB3 — Sciences mathématiques

2026–2027

TABLE DES MATIÈRES

1	Logique, langage et calcul	2
1.1	Introduction informel	2
2	Langages et automates	4
2.1	Automates finis	4
2.2	Théorème de Klein	5
2.2.1	Expressions régulières	6
2.2.2	Automates non déterministes	8

1. LOGIQUE, LANGAGE ET CALCUL

1.1 INTRODUCTION INFORMEL

On veut comprendre quels types de raisonnements on peut automatiser, on doit donc s'intéresser à la notion d'algorithme. Dont le fonctionnement est illustré ci-dessous :



Tout l'intérêt sera de savoir si on peut trouver un algorithme qui répond à une question en temps fini (qui ne boucle pas) . Il nous faut donc formaliser toutes ces notions.

Définition 1.1.1 (mot). *C'est un ensemble de symboles finis, que l'on peut concaténer pour former des chaînes de caractères.*

Définition 1.1.2. *Un dictionnaire Σ^* est l'ensemble de tout les mots finis (dont le mot vide) formé par les lettres (symboles) d'un ensemble nommé alphabet et noté Σ*

Un **Input** d'un algorithme doit être finiment représentable. Par exemple $\mathbb{N}$ est un input infini mais représenté par un symbole. En fait, on a besoin d'une fonction d'encodage

$$x \mapsto \text{enc}(x) \in \Sigma^*$$

Etant donné un mot $w \in \Sigma^*$, décider le problème $\mathcal{P}$ revient à décider si $w \in \mathcal{L}_{\mathcal{P}}$ où

$$\mathcal{L}_{\mathcal{P}} := \{x | x \text{ est une instance positive de } \mathcal{P}\}$$

Pour être clair, une instance positive est l'ensemble des inputs dont la question a été répondu favorablement.

Exemple 1.1.3. Etant donné un mot binaire, contient- il le facteur "101"? On est avec l'alphabet $\Sigma = \{0, 1\}$ et $\mathcal{L}_{101} = \{w \in \Sigma^* | w \text{ contient le facteur "101"}\}$ On peut réfléchir à un algorithme pour déterminer si un mot appartient à $\mathcal{L}_{101}$. La notion de mémoire apparaît!

Avec ce même formalisme, nous pouvons répondre à des problèmes bien plus compliqués.

Définition 1.1.4. *On définit un système formel S comme la réunion d'un alphabet, syntaxe, sémantique et ect..*

Le premier problème est :

Avec φ, π dans un système S , où φ est un énoncé et π est une tentative de preuve. On veut déterminer

$$V_S = \{\langle \varphi, \pi \rangle \mid \pi \text{ est une preuve valide de } \varphi \text{ dans } S\}$$

En générale, déterminer si une preuve est correcte est décidable.

Le second problème encore plus compliqué est :

Sous les mêmes hypothèses déterminons

$$T_S = \{\varphi \mid \exists \text{ une preuve } \pi \text{ dans } S\}$$

Il faut distinguer qu'en pratique on peut même s'il n'existe pas de preuve avec certains inputs sur d'autres input on pourrait trouver une preuve. Mais le fait que déterminer ce langage soit encore plus compliqué est étroitement lié au théorème de complétude de Godel. Il faut distinguer la **vérité** et la **démontrabilité** parfois les notions sémantiques et la notion syntaxique d'une preuve ne peuvent pas toujours correspondre.

2. LANGAGES ET AUTOMATES

2.1 AUTOMATES FINIS

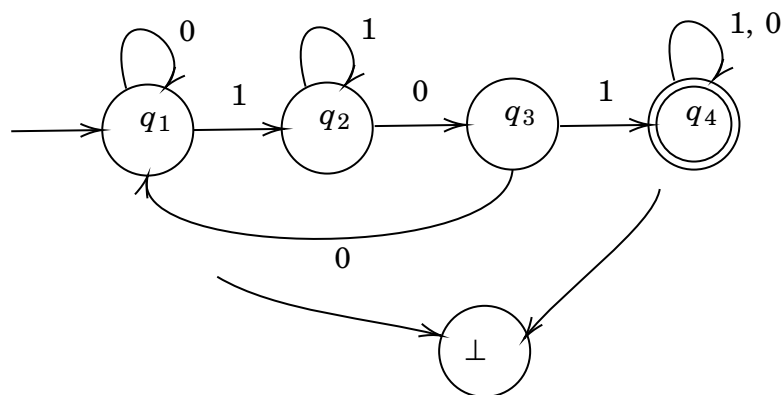
on a vu le **problème** suivant :

Etant donné un mot binaire, contient- il le facteur "101"?

On est avec l'alphabet $\Sigma = \{0, 1\}$ et $\mathcal{L}_{101} = \{w \in \Sigma^* | w \text{ contient le facteur "101"}\}$ On peut réfléchir à un algorithme pour déterminer si un mot appartient à $\mathcal{L}_{101}$. On veut donc trouver un algorithme pour reconnaître, décider oui ou non en s'arrêtant, si un mot appartient à ce langage. Nous allons construire un automate.

Définition 2.1.1 (informel). *Un automate c'est une machine qui représente un langage il prend en entrée un mot lettre après lettre et dira s'il est accepté ou non.*

Pour notre problème on construit l'automate suivant :



On peut compléter avec un état poubelle

On peut tester l'automate sur différents mots et vérifier si l'état final est q_4 qui est acceptant. Dans ce cas le mot appartiendra au langage de l'automate.

Remarque 2.1.2. On peut remarquer que $w_1 = 11$ et $w_2 = 0001$ finissent sur le même état et en fait on verra qu'ils peuvent être vus comme équivalents en terme de continuation qu'on verra plus tard.

En fait, un automate est $\mathcal{L}(\mathcal{A}) \subseteq \Sigma^*$ qui prend $w \in \Sigma^*$ et qui accepte si $w \in \mathcal{L}(\mathcal{A})$ et le rejette sinon.

A présent, formalisons les différents concepts que nous avons rencontré.

Définition 2.1.3 (Automate fini déterministe (DFA)). *Voici des définitions plus formelles.*

- (i) *On définit un mot comme une suite finie de lettres (symboles) sur un alphabet (ensemble de symbole) Σ .*
- (ii) *Un langage $\mathcal{L}$ est un sous-ensemble de tous les mots $\mathcal{L} \subseteq \Sigma^*$*
- (iii) *Soient Σ un alphabet, Q un ensemble fini (d'état), $q_0 \in Q$ un état initial, $F \subseteq Q$ l'ensemble des états acceptant et $\delta : Q \times \Sigma \rightarrow Q$ une fonction dites de transition. On définit un automate comme un tuple*

$$\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$$

- (iv) *On définit un Run pour $w = a_1 \cdots a_n \in \Sigma^*$ comme un séquence d'états*

$$\rho = r_0 r_1 \cdots r_n$$

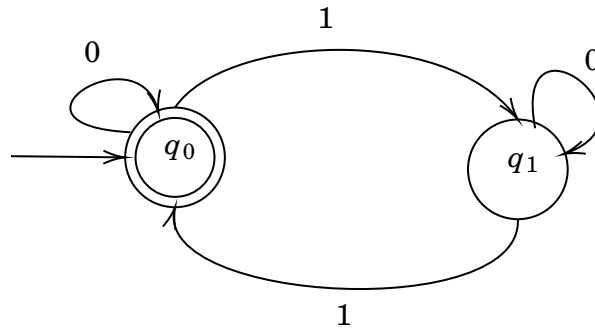
tel que $r_0 = q_0$ et $\forall 1 \leq i \leq n, r_i = \delta(r_{i-1}, a_i)$

- (v) *On dit que w est acceptant si $r_n \in F$. De plus, on pose*

$$\mathcal{L}(\mathcal{A}) = \{w \in \Sigma^* | w \text{ est accepté par } \mathcal{A}\}$$

- (vi) *Un automate est déterministe si pour chaque mot il existe un unique run.*

Exemple 2.1.4. Considérons le langage $\mathcal{L} = \{w \in \{0, 1\}^* | \text{le nombre de 1 est pair}\}$



Exemple 2.1.5. pour $\mathcal{L} = \{w \in \{0, 1\}^* | \text{qui dit s'il y a autant de 0 que de 1}\}$ Il n'est pas possible de trouver un automate pour cela.

2.2 THÉORÈME DE KLEIN

Dans cette section, nous allons étudier le théorème de Kleen. Pour cette raison nous aurons besoins de différentes notions.

Définition 2.2.1 (DFA-rec). *On dit qu'un langage $\mathcal{L}$ est reconnu par un DFA si il existe un automate qui accepte tous les mots du langage.*

Théorème 2.2.2. *Pour tout langage $\mathcal{L}$ on a l'équivalence entre $\mathcal{L}$ est reconnu par un DFA et que $\mathcal{L}$ est défini par une RegExp. Si ces propositions sont vraies, on dit que $\mathcal{L}$ est régulier.*

2.2.1 EXPRESSIONS RÉGULIÈRES

Soient $\mathcal{L}_1$ et $\mathcal{L}_2$ deux langages sur un même alphabet (non nécessaire), on définit :

- $\mathcal{L}_1 \cup \mathcal{L}_2 = \{w | w \in \mathcal{L}_1 \vee w \in \mathcal{L}_2\}$
- $\mathcal{L}_1 \circ \mathcal{L}_2 = \{w_1 w_2 | w_1 \in \mathcal{L}_1 \wedge w_2 \in \mathcal{L}_2\}$
- $\mathcal{L}_1^* = \{w_1 \dots w_k | k \geq 0 \wedge \forall 1 \leq i \leq k, w_i \in \mathcal{L}_1\}$

Exemple 2.2.3. Considérons $\mathcal{L}_1 = \{ \text{Black, Yellow} \}$ et $\mathcal{L}_2 = \{ \text{Night, Submarine} \}$

- $\mathcal{L}_1 \cup \mathcal{L}_2 = \{ \text{Black, Yellow, Night, Submarine} \}$
- $\mathcal{L}_1 \circ \mathcal{L}_2 = \{ \text{BlackNight, BlackSubmarine, YellowNight, YellowSubmarine} \}$
- $\mathcal{L}_1^* = \{ \varepsilon, \text{Black, Yellow, BlackYellow, ...} \}$

Soit l'alphabet Σ une expression régulière E est formée de manière inductive par

$$E := \emptyset \mid \varepsilon \mid a \mid E + E' \mid EE' \quad a \in \Sigma$$

Exemple 2.2.4. Voici quelques exemples :

- $\mathcal{L}(\emptyset) = \emptyset$
- $\mathcal{L}(\varepsilon) = \{\varepsilon\}$
- $\mathcal{L}(E_1 + E_2) := \mathcal{L}(E_1) \cup \mathcal{L}(E_2)$
- $\mathcal{L}(E_1 E_2) = \mathcal{L}(E_1) \circ \mathcal{L}(E_2)$
- $\mathcal{L}(E^*) = \mathcal{L}(E)^*$
- $\mathcal{L}(E_1 \emptyset) = \emptyset$.
- $\mathcal{L}(\emptyset^*) = \{\varepsilon\}$
- Dés lors posons $\Sigma = \{a, b, c\}$
 - $\mathcal{L}((a|b|c)^*) = \Sigma^*$
 - $\mathcal{L}(a(b|c)^*a) = \{aba, aca, abba, abca, acba, acca, \dots\}$

Remarque 2.2.5. Ainsi le langage du premier automates est $\mathcal{L}(\mathcal{A}) = \mathcal{L}((0|1)^*101(0|1)^*)$

Pour être clair, la **la priorité des opérations** est naturellement :

- (i) E^*
- (ii) EE'
- (iii) $E + E'$

Notons que le théorème de Kleen établit donc une équivalence entre deux objets qui pourtant semblent bien différents.

REGEXP IMPLIQUE DFA

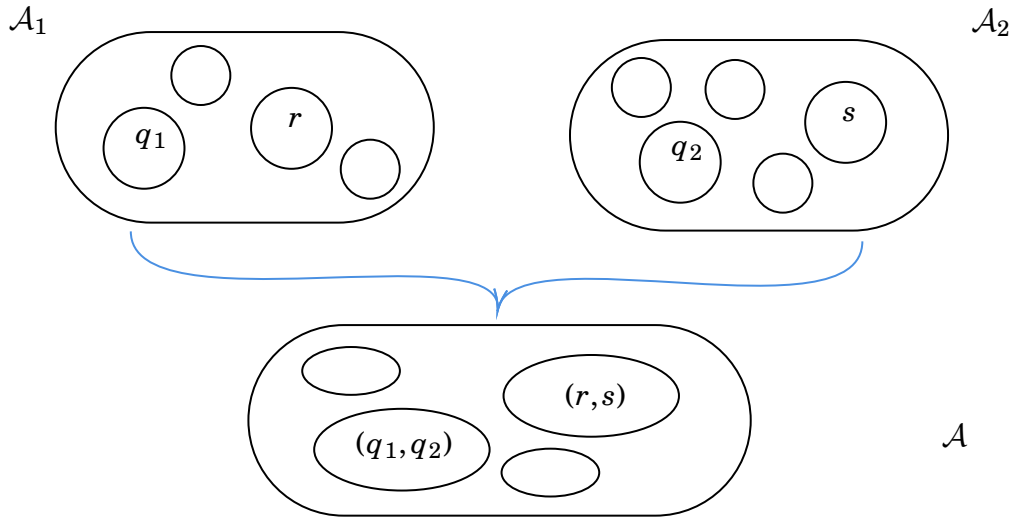
Nous allons d'abord nous intéresser à la direction RegExp implique DFA. Pour cela, analysons d'abord **les propriétés de clôture** des langages reconnu par des DFA pour les opérations régulières.

Lemme 2.2.6. Si $\mathcal{L}_1$ et $\mathcal{L}_2$ sont des DFA-rec alors $\mathcal{L}_1 \cup \mathcal{L}_2$ est DFA-rec.

Démonstration. Débutons la preuve :

Soient $\mathcal{A}_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ rec $\mathcal{L}_1$ et $\mathcal{A}_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ rec $\mathcal{L}_2$

Nous voulons construire $\mathcal{A} = (Q, \Sigma, \delta, q, F)$ tel que $\mathcal{L}(\mathcal{A}) = \mathcal{L}_1 \cup \mathcal{L}_2$. Pour illustrer l'idée :



Posons $Q = Q_1 \times Q_2$ et $q_0 = \langle q_1, q_2 \rangle$ et $\delta(\langle r, s \rangle, a) = \langle \delta_1(r, a), \delta_2(s, a) \rangle$ Pour F prendre $F_1 \times F_2$ ne convient pas. Cependant $F = (Q_1 \times F_2) \cup (F_1 \times Q_2)$ convient car nous voulons que dès que un des deux langages acceptent un mot c'est accepté. Pour s'en convaincre,

Montrons que

- $\mathcal{L}(\mathcal{A}) \subseteq (\mathcal{L}(\mathcal{A}_1) \cup \mathcal{L}(\mathcal{A}_2))$.

Prenons un mot w accepté par $\mathcal{A}$; on doit montrer qu'il est aussi accepté par $\mathcal{A}_1$ ou $\mathcal{A}_2$. Par définition de $w \in \mathcal{L}(\mathcal{A})$, le run de w dans $\mathcal{A}$ s'arrête sur un état de F , c'est-à-dire un couple $\langle r, s \rangle$ tel que $r \in F_1$ ou $s \in F_2$. Sans perte de généralité, supposons que $r \in F_1$. Le run dans $\mathcal{A}$ projeté dans $\mathcal{A}_1$ est un run de $\mathcal{A}_1$ qui finit en $r \in F_1$. Donc $\mathcal{A}_1$ accepte w et $w \in \mathcal{L}_1$. A fortiori, $w \in \mathcal{L}_1 \cup \mathcal{L}_2$.

- $\mathcal{L}_1 \cup \mathcal{L}_2 \subseteq \mathcal{L}(\mathcal{A})$.

Prenons un mot w accepté par $\mathcal{A}_1$ (sans perte de généralité). Son run dans $\mathcal{A}_1$ peut être reproduit dans $\mathcal{A}$ par construction, et finit dans un état $\langle r, s \rangle$ où $r \in F_1$ (par hypothèse). Par définition de F , on a donc que $\mathcal{A}$ accepte aussi w , et $w \in \mathcal{L}(\mathcal{A})$. □

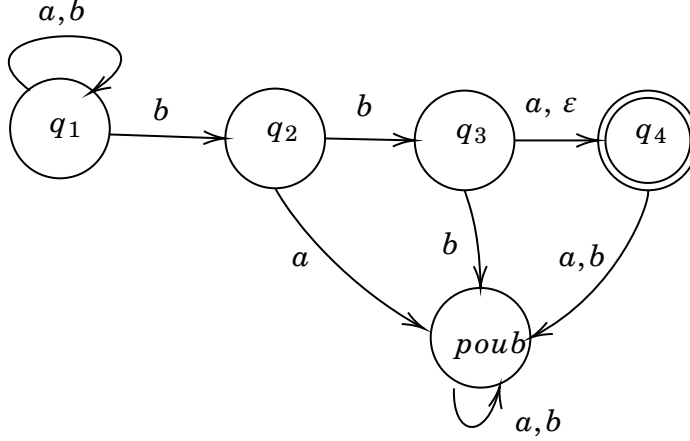
Lemme 2.2.7 (Clôture pour la concaténation). Si $\mathcal{L}_1$ et $\mathcal{L}_2$ sont deux langages DFA-rec alors $\mathcal{L}_1 \circ \mathcal{L}_2$ est DFA-rec

Le lemme précédent semble compliqué à prouver (voir notes manuscrites pour plus de détails). En effet, Il apparait que pour construire un tel objet nous aurions besoin de connaître le "future" d'un mot. Pour contourner ce problème nous allons, à présent, nous intéresser aux automates non déterministes.

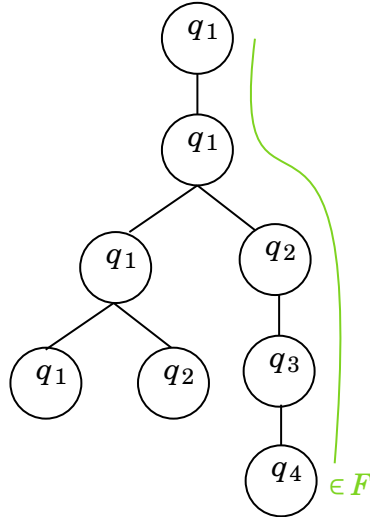
2.2.2 AUTOMATES NON DÉTERMINISTES

Abordons, d'abord, cette notion par un exemple.

Exemple 2.2.8 (NFA epsilon). Posons $\Sigma = \{a, b\}$ considérons l'automate suivant :



Dans ce cas, il y a plusieurs runs par mot s'il existe une run où le mot est accepté alors le mot est accepté et que tout le mot soit lu. Par exemple avec le mot $w = abba$ ça donne :



Notons que $\mathcal{L}(\mathcal{A}) = \mathcal{L}((a|b)^*bb(a|\epsilon))$

Définissons formellement les $\text{NFA}\epsilon$:

Définition 2.2.9 (Automate fini non-déterministe avec ϵ -transitions). Un $\text{NFA}\epsilon$ est un tuple $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ où

- Q est un ensemble fini d'états,
- Σ est un alphabet fini,
- $q_0 \in Q$ est l'état initial,
- $F \subseteq Q$ est un ensemble d'états acceptants,
- $\delta : Q \times \Sigma_\epsilon \rightarrow 2^Q$ est la fonction de transition.

Définition 2.2.10 (Run dans un NFA ϵ). *Un run pour un mot $w \in \Sigma^*$ est une séquence d'états $\rho = r_0 \cdots r_m$ associée à une séquence de symboles $a_1 \cdots a_m \in \Sigma_\epsilon^*$ telle que*

- $r_0 = q_0$,
- $\forall 1 \leq i \leq m, r_i \in \delta(r_{i-1}, a_i)$,
- $w = a_1 \cdot a_2 \cdots a_m$.

Définition 2.2.11 (Acceptance dans un NFA ϵ). *Un mot w est accepté s'il existe un run $\rho = r_0 \cdots r_m$ sur ce mot tel que $r_m \in F$.*

Trois visions du non-déterminisme

- (i) **arbres.** Nous créons un arbre d'exécution avec un branchement pour chaque choix non-déterministe : nous acceptons si une des branches accepte.
- (ii) **calcul parallèle.** À chaque choix non-déterministe, nous faisons un « fork » : nous lançons un nouveau processus qui fonctionne en parallèle. Nous acceptons si un des processus accepte.
- (iii) **Magique** Nous « devinons » le choix correct (celui qui amènera à OUI) et nous allons de ce côté.

Proposition 2.2.12. *On peut transformer tout NFA ϵ $\mathcal{A}_\epsilon$ en NFA (sans ϵ transition) $\mathcal{A}$ équivalent. Autrement dit,*

$$\mathcal{L}(\mathcal{A}_\epsilon) = \mathcal{L}(\mathcal{A})$$

Démonstration.

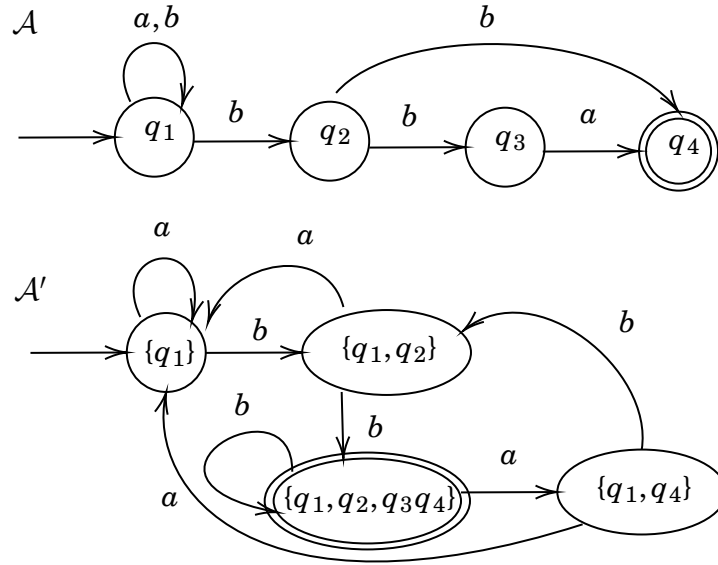
□

Théorème 2.2.13. *Pour tout NFA $\mathcal{A}$, il existe un DFA $\mathcal{A}'$ tel que*

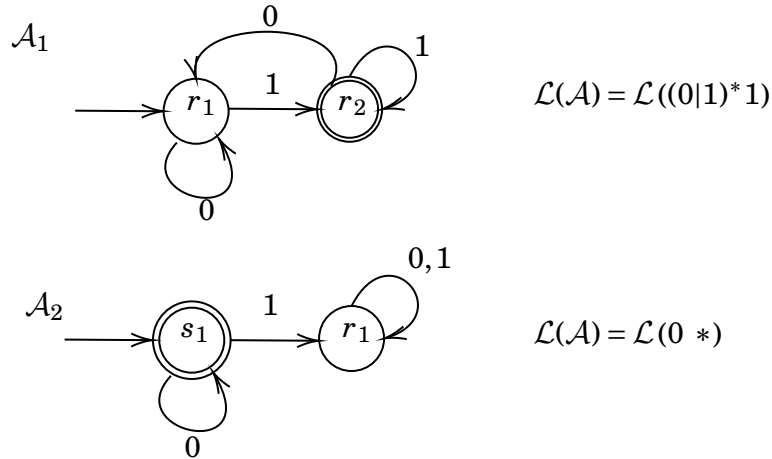
$$\mathcal{L}(\mathcal{A}) = \mathcal{L}(\mathcal{A}')$$

Démonstration. Soit $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ un NFA. Nous allons construire un DFA $\mathcal{A}' = (Q', \Sigma, \delta', q'_0, F')$ tel que $Q' = 2^Q$ et $\delta'(R, a) = \{q \in Q \mid \exists r \in R, q \in \delta(r, a)\}$. □

Exemple 2.2.14. Considérons le NFA suivant :



Exemple 2.2.15. Considérons le NFA suivant :



Proposition 2.2.16. Si $\mathcal{A}$ est un NFA, alors il existe un NFA $\mathcal{A}'$ tel que $\mathcal{L}(\mathcal{A}) = \mathcal{L}$ et $\mathcal{L}(\mathcal{A}') = \mathcal{L}^*$.

Démonstration. Soit $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ un NFA. On construit le NFA $\mathcal{A}' = (Q', \Sigma, \delta', q'_0, F')$ de la manière suivante :

- $Q' = Q \cup \{q'_0\}$ où q'_0 est un nouvel état initial.
- $\delta'(q'_0, a) = \delta(q_0, a)$ pour tout $a \in \Sigma$ et $\delta'(q'_0, \epsilon) = F$.
- $\delta'(q, a) = \delta(q, a)$ pour tout $q \in Q$ et $a \in \Sigma$.
- $F' = F$.

Ainsi, $\mathcal{A}'$ accepte toutes les chaînes qui peuvent être obtenues en concaténant zéro ou plusieurs chaînes acceptées par $\mathcal{A}$, ce qui correspond à l'étoile de Kleene de $\mathcal{L}(\mathcal{A})$. Par conséquent, $\mathcal{L}(\mathcal{A}') = \mathcal{L}^*$. $\square$

Exemple 2.2.17. Considérons le NFA $\mathcal{A}$ suivant : qui est $\mathcal{L}(\mathcal{A}) = \mathcal{L}((a|b)^*bb(\varepsilon|a))$

Nous avons E un RegExp on veut à présent construire un NFA $\mathcal{A}$ tel que $\mathcal{L}(\mathcal{A}) = \mathcal{L}(E)$.

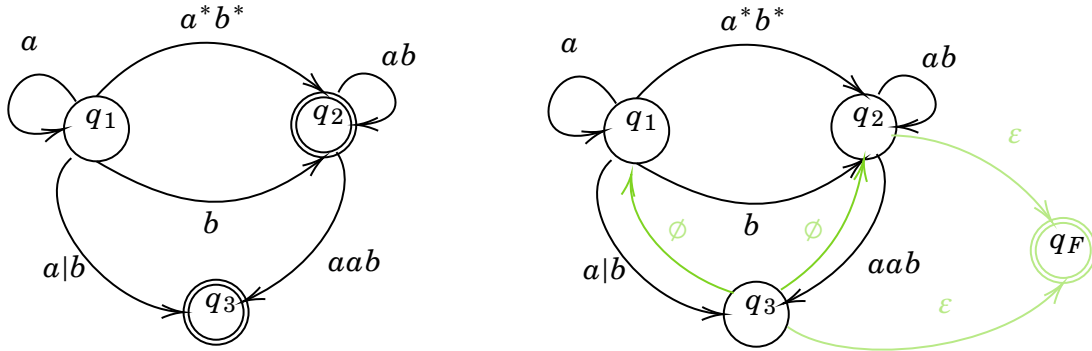
- Si E est atomique.

- $E = a$ où $a \in \Sigma$
- $E = \varepsilon$
- $E = \emptyset$

En fait nous avons déjà vu comment construire un NFA pour ces trois cas.

On veut construire un NFA pour $(a|ab)^*$: **Atomiques** : à faire notes **Composite** : à faire voir notes

DFA $\rightarrow$ RegExp



Soit $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ un automate en forme spéciale.

avant ça à retravailler sur les notes c'est un peu brouillon

V. FORME CANONIQUE ET MINIMISATION

Nous avons vu que les expressions régulières (RegExp) n'ont pas de forme canonique, ce qui rend leur comparaison difficile.

Mais, grâce au théorème de Kleene, nous savons qu'une RegExp n'est qu'une des représentations possibles d'un langage régulier : nous avons aussi accès aux automates. Nous allons voir que *ces derniers admettent une forme canonique*. Ils seront donc à privilégier pour comparer (et, plus généralement, manipuler) des langages.

Idée. Pour un langage $\mathcal{L}$, on va classer les mots $w \in \Sigma^*$ en fonction des continuations possibles et de leur impact sur l'appartenance à $\mathcal{L}$. Deux mots w_1 et w_2 qui sont équivalents vis-à-vis des continuations seront dans la même classe.

Anil Nerode (1932–), logicien américain : logique, langages, complexité, etc.

Définition 2.2.18 (Congruence de Nerode). Pour un mot $w \in \Sigma^*$ et un langage $\mathcal{L} \subseteq \Sigma^*$, on définit

$$w^{-1}\mathcal{L} = \{w' \in \Sigma^* \mid ww' \in \mathcal{L}\},$$

l'ensemble des continuations w' de w telles que $ww' \in \mathcal{L}$.

On définit la relation d'équivalence $\sim_{\mathcal{L}} \subseteq \Sigma^* \times \Sigma^*$ par

$$w_1 \sim_{\mathcal{L}} w_2 \iff w_1^{-1}\mathcal{L} = w_2^{-1}\mathcal{L}.$$

On va en tirer une caractérisation de l'automate *minimal* pour un langage $\mathcal{L}$. C'est important quand on utilise les automates comme structure de données.

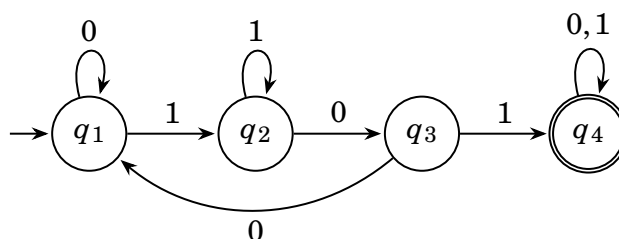
John Myhill (1923–1987), mathématicien anglais. Pionnier en théorie des langages, théorie des ensembles, calculabilité, etc.

Théorème 2.2.19 (Myhill–Nerode).

- (i) Un langage $\mathcal{L}$ est régulier ssi $\sim_{\mathcal{L}}$ a un nombre fini de classes d'équivalence.
- (ii) Ce nombre est égal au nombre minimal d'états d'un DFA reconnaissant $\mathcal{L}$.
- (iii) Ce DFA minimal est unique à isomorphisme près.

Remarque 2.2.20. On obtient ainsi une *troisième caractérisation* de la régularité d'un langage.

Exemple 2.2.21. Considérons l'automate $\mathcal{A}$ suivant :



Notre automate pour $(0|1)^* 101(0|1)^*$ est-il minimal?

Considérons

$$w_1 = \varepsilon, \quad w_2 = 1, \quad w_3 = 10, \quad w_4 = 101.$$

Chaque w_i est celui qui nous a inspiré pour créer q_i . On a bien $w_i \not\sim_{\mathcal{L}} w_j$ pour tout $i \neq j$:

- $w_1 \not\sim_{\mathcal{L}} w_2$ car $w_1 01 \notin \mathcal{L}$ et $w_2 01 \in \mathcal{L}$;
- $w_2 \not\sim_{\mathcal{L}} w_3$ car $w_2 1 \notin \mathcal{L}$ et $w_3 1 \in \mathcal{L}$;
- $w_3 \not\sim_{\mathcal{L}} w_4$ car $w_3 \varepsilon \notin \mathcal{L}$ et $w_4 \varepsilon \in \mathcal{L}$;
- $w_1 \not\sim_{\mathcal{L}} w_3$ car $w_1 1 \notin \mathcal{L}$ et $w_3 1 \in \mathcal{L}$;
- $w_1 \not\sim_{\mathcal{L}} w_4$ car $w_1 \varepsilon \notin \mathcal{L}$ et $w_4 \varepsilon \in \mathcal{L}$;
- $w_2 \not\sim_{\mathcal{L}} w_4$ car $w_2 \varepsilon \notin \mathcal{L}$ et $w_4 \varepsilon \in \mathcal{L}$.

Nous avons donc déjà l'automate minimal.

Remarque 2.2.22. Les états de l'automate minimal correspondent aux classes de $\sim_{\mathcal{L}}$. L'existence d'un automate minimal (à isomorphisme près) est intéressante, mais on voudrait aussi pouvoir le calculer.

MINIMISATION DE DFA

Proposition 2.2.23. *Étant donné un DFA $\mathcal{A}$, on peut construire un DFA minimal $\mathcal{A}'$ tel que $\mathcal{L}(\mathcal{A}') = \mathcal{L}(\mathcal{A})$ en temps polynomial.*

Remarque 2.2.24. Cela deviendra plus clair dans le chapitre « Complexité », mais c'est une bonne nouvelle. On ne verra pas l'algorithme ; allez voir Esparza et Blondin si cela vous intéresse.

QUID DES NFA ?

Malheureusement, il n'y a pas d'unicité de l'automate minimal. Considérons $\mathcal{L} = \mathcal{L}(aa^*)$.



Nous avons deux automates minimaux qui ne sont pas isomorphes ! Cela rend les choses beaucoup plus compliquées.

Théorème 2.2.25 (Complexité de la minimisation de NFA). *Soient $\mathcal{A}$ un NFA et $k \geq 1$. Décider s'il existe un NFA $\mathcal{A}'$ avec au plus k états tel que $\mathcal{L}(\mathcal{A}') = \mathcal{L}(\mathcal{A})$ est un problème PSPACE-complet.*

Remarque 2.2.26. Ça, par contre, c'est une mauvaise nouvelle ! En pratique, cela demande un temps exponentiel ; on ne sait pas si un algorithme polynomial existe (et on pense que non, sinon $P = PSPACE$).

VI. CARACTÉRISATION LOGIQUE

Pour faire le lien avec IntLog, considérons une dernière caractérisation des langages réguliers, via la logique. Il existe deux philosophies de description des langages :

RegExp	Logique
Description <i>générationnelle</i> : « comment créer un mot du langage ».	Description <i>déclarative</i> : « propriété à satisfaire pour appartenir au langage ».

En fonction du langage, l'une ou l'autre description peut être plus facile à comprendre.

Exemple 2.2.27. Le langage des mots ayant un nombre pair de a et un nombre pair de b :

$$\mathcal{L} = \{w \in \{a, b\}^* \mid \#a \bmod 2 = 0 \wedge \#b \bmod 2 = 0\}$$

versus

$$\mathcal{L}\left([aa \mid bb \mid (ab \mid ba)(aa \mid bb)^*(ba \mid ab)]^*\right).$$

Nous allons utiliser la *logique des prédicats sur les mots*, qui est la logique du *premier ordre* : les quantificateurs ne portent que sur les « variables d'individus », jamais sur des propriétés, des prédicats ou des fonctions.

Fixons un alphabet $\Sigma = \{a, b\}$. On distingue deux types de propriétés atomiques :

- (i) $P_a(x)$ ou $P_b(x)$, où x est une variable désignant une position au sein du mot. Par exemple, $P_b(7)$ est le prédicat qui est vrai si la lettre à la 7^e position est « b ».
- (ii) $x < y$, où x et y sont deux variables de position.

Exemple 2.2.28. $\mathcal{L}(a^*b^*)$: si on a un « b » dans une position, alors on en a aussi dans toutes les suivantes. En logique des prédicats :

$$\forall x \forall y [(P_b(x) \wedge x < y) \rightarrow P_b(y)].$$

Exemple 2.2.29.

$$\forall x \forall y [(P_a(x) \wedge x < y \wedge P_a(y)) \rightarrow \exists z (x < z \wedge z < y \wedge P_b(z))]$$

signifie : entre deux « a », il y a au moins un « b ».